

Nomes e RMs

Nome: Gabriel Camarosani Gouvea Gonçalves da Silva RM: 569189

Nome: Bruno Yudi Moritaka Kanashiro RM: 571776

Nome: Lucas Seiji Hummel. RM: 569673

Nome: Gustavo Lima Andrade Santos RM: 571709

Nome: Pedro Souza Castro RM: 569311

Nome: Lucas Barreto Santana RM: 573149

Parte 1 - Recursividade: somatório

1. Indique o caso base da função.

R: `if (n == 0){return 0;}`

2. Indique a chamada recursiva e explique como o problema diminui.

R: `return n + somatoria(n - 1);`

O problema diminui chamando a si mesmo, com o $n-1$, ou seja, o caso anterior, e segue nessa diminuição até chegar em zero.

3. Desenhe a sequência de chamadas para somatorio(5).

R:

`somatoria(5) = 5 + somatoria(4)`

`somatoria(4) = 4 + somatoria(3)`

`somatoria(3) = 3 + somatoria(2)`

`somatoria(2) = 2 + somatoria(1)`

`somatoria(1) = 1 + somatoria(0) return 0 -> 1 -> 3 -> 6 -> 10 -> 15`

`somatoria(0) = 0`

4. Determine a complexidade temporal e a complexidade de memória da recursão

R: A complexidade temporal e a complexidade da memória são $O(n)$, pois, para complexidade temporal, ele realiza uma chamada recursiva para cada 'n', e para complexidade da memória, existem $n+1$ elementos na pilha de execução

Parte 2 - Recursividade aplicada a vetor

1. Qual deve ser o caso base de somaVetor?

R: `if (n == 0){return 0;}`

2. Como o parâmetro *n* pode representar o tamanho do problema restante?

R: 'n' pode representar o tamanho do problema restante, pois 'n' é utilizado para pegar o valor dentro da lista, sem ele não é possível utilizar-la, então ele representa o quanto que a recursividade vai ser aplicada

3. Explique por que somaVetor e maiorVetor possuem complexidade $O(n)$.

R: As funções somaVetor e maiorVetor possuem complexidade $O(n)$, pois em cada chamada é realizada uma quantidade constante de operações, realizando uma por elemento, de forma que essas operações crescem linearmente

Parte 3 - Do ponteiro ao nó da lista

1. O que representa o campo proximo?

R: Representa o lugar da memória do próximo No

2. Explique a diferença entre No, No * e No *proximo.

R: O No representa o valor e o endereço do próximo No
O No * acessa o valor armazenado no endereço que o ponteiro aponta
O No *proximo significa "ponteiro para outro No"

3. Por que utilizamos o operador -> para acessar valor e proximo quando temos um ponteiro?

R: Pois o operador -> acessa campos quando temos um ponteiro para a struct. , já proximo é um ponteiro para outra struct

Parte 4 - Criando uma lista e inserindo no início

1. *Explique o que acontece com os ponteiros ao inserir 30, depois 20 e depois 10.*

R: A variável `inicio` é atualizada a cada inserção, apontando sempre para o primeiro nó da lista, ou seja, 1o aponta para NULL, depois para 30 e depois 20

2. *Desenhe a lista resultante*

R: `inicio -> [10 | *] -> [20 | *] -> [30 | NULL]`

3. *Por que inserir no início é $O(1)$?*

R: Porque a inserção no início exige apenas criar um novo nó, ajustar seu ponteiro para apontar para o início atual e atualizar o ponteiro `inicio`

Parte 5 - Imprimindo a lista

1. *Por que não devemos alterar o ponteiro `inicio` durante a impressão?*

R: Não devemos alterar o `inicio` porque ele mantém a referência para o primeiro nó da lista. Se utilizássemos esse próprio ponteiro para percorrer os nós, ele avançaria até NULL e perderíamos a referência original ao começo da lista. Por isso, criamos o ponteiro auxiliar `atual`, que pode percorrer a lista sem modificar `inicio`

2. *O que a instrução `atual = atual->proximo` faz?*

R: A instrução faz o ponteiro `atual` deixar de apontar para o nó corrente e passar a apontar para o próximo nó indicado pelo campo `proximo`. Assim, o programa consegue percorrer sequencialmente todos os nós até encontrar NULL, que representa o fim da lista.

3. *Qual é a complexidade para imprimir uma lista com n nós?*

R: A complexidade temporal é $O(n)$, porque a função percorre cada um dos n nós uma vez para exibir seu valor. Portanto, conforme o número de nós aumenta, a quantidade de operações de percurso cresce linearmente

Parte 6 - Busca sequencial na lista

1. Qual é o melhor caso da busca?

R: O melhor caso da busca é $O(1)$, quando o valor buscado é o primeiro na lista.

2. Qual é o pior caso da busca?

R: O pior caso da busca é $O(n)$, quando o valor buscado é o último da lista.

3. Por que a lista não permite acesso direto como vetor[i]?

R: Porque os elementos de uma lista estão espalhados pela memória, o que força o algoritmo a percorrer cada elemento até achar o alvo, enquanto os elementos do vetor estão em blocos de memórias contíguos, o que permite o algoritmo a achar o valor alvo usando uma fórmula matemática.

Parte 7 - Análise de complexidade

Operação	Melhor caso	Pior Caso	Justificativa
Criar lista vazia	$O(1)$	$O(1)$	O tempo de execução é constante pois só é necessário atribuições para criar uma lista vazia.
Inserir no início	$O(1)$	$O(1)$	A inserção no início não precisa deslocar os outros elementos por usar ponteiros, então seu tempo de execução é constante.
Mostrar primeiro elemento	$O(1)$	$O(1)$	O programa sempre só precisará fazer uma operação para mostrar o primeiro elemento, o que é constante.
Imprimir lista	$O(1)$	$O(n)$	$O(1)$ se a lista estiver vazia ou ter um elemento e $O(n)$

			se tiver mais de um.
Buscar valor	$O(1)$	$O(n)$	$O(1)$ se o valor alvo for o primeiro buscado e $O(n)$ se for o último ou não existir.

Parte 8 - Desafio integrador

//Nomes e RMs

//Nome: Gabriel Camarosani Gouvea Gonçalves da Silva RM: 569189

//Nome: Bruno Yudi Moritaka Kanashiro RM: 571776

//Nome: Lucas Seiji Hummel. RM: 569673

//Nome: Gustavo Lima Andrade Santos RM: 571709

//Nome: Pedro Souza Castro RM: 569311

//Nome: Lucas Barreto Santana RM: 573149

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
typedef struct No No;
```

```
struct No {
    int valor;
    No *proximo;
};
```

```
No *inserirInicio(No *inicio, int valor) {
    No *novo;
```

```
    // ALOCAÇÃO DO NÓ
```

```
    novo = (No *) malloc(sizeof(No));
```

```
    if (novo == NULL) {
        printf("Erro ao alocar memoria.\n");
        return inicio;
    }
```

```
    novo->valor = valor;
```

```
    // LIGAÇÃO COM O PRÓXIMO NÓ
```

```
    novo->proximo = inicio;
```

```

    return novo;
}

void imprimirLista(No *inicio) {
    No *atual = inicio;

    // CASO BASE: a lista está vazia, portanto não há nós para percorrer.
    if (inicio == NULL) {
        printf("A lista esta vazia.\n");
        return;
    }

    printf("Lista: ");

    while (atual != NULL) {
        printf("%d -> ", atual->valor);
        atual = atual->proximo;
    }

    printf("NULL\n");
}

int buscar(No *inicio, int valor) {
    No *atual = inicio;

    // BUSCA: percorre a lista procurando o valor informado.
    while (atual != NULL) {
        if (atual->valor == valor) {
            return 1;
        }

        atual = atual->proximo;
    }

    return 0;
}

void liberarLista(No *inicio) {
    No *atual = inicio;
    No *proximo;

    while (atual != NULL) {
        proximo = atual->proximo;
        free(atual);
        atual = proximo;
    }
}

```

```

int main(void) {
    No *inicio = NULL;
    int opcao;
    int valor;

    do {
        printf("\n===== LISTA ENCADEADA =====\n");
        printf("1 - Inserir valor no inicio\n");
        printf("2 - Mostrar lista\n");
        printf("3 - Buscar valor\n");
        printf("4 - Mostrar primeiro elemento\n");
        printf("0 - Sair\n");
        printf("Opcao: ");
        scanf("%d", &opcao);

        switch (opcao) {
            case 1:
                printf("Valor: ");
                scanf("%d", &valor);

                inicio = inserirInicio(inicio, valor);

                printf("Valor inserido com sucesso.\n");
                break;

            case 2:
                imprimirLista(inicio);
                break;

            case 3:
                printf("Valor para buscar: ");
                scanf("%d", &valor);

                if (buscar(inicio, valor)) {
                    printf("Valor encontrado na lista.\n");
                } else {
                    printf("Valor nao encontrado na lista.\n");
                }

                break;

            case 4:
                if (inicio == NULL) {
                    printf("A lista esta vazia.\n");
                } else {
                    printf("Primeiro elemento: %d\n", inicio->valor);
                }
        }
    }
}

```

```

        break;

    case 0:
        printf("Encerrando o programa...\n");
        break;

    default:
        printf("Opcao invalida.\n");
    }

} while (opcao != 0);

liberarLista(inicio);

return 0;
}

```

Parte 9 - Questões de compreensão

1. Qual é a função do caso base em um algoritmo recursivo?

O caso base determina quando a recursão deve parar, evitando chamadas infinitas.

2. O que acontece quando uma recursão não converge para um caso base?

Ela continua fazendo chamadas até ocorrer um erro, geralmente por stack overflow

3. Qual é a função de NULL em uma lista encadeada?

Indicar que não existe um próximo nó, ou seja, que chegou ao final da lista.

4. Explique por que o nome “encadeada” faz sentido para essa estrutura.

Porque cada nó possui uma referência para o próximo nó, formando uma sequência de elementos encadeados.

5. Qual a diferença entre acessar vetor[i] e chegar ao i-ésimo nó de uma lista?

No vetor, o elemento pode ser acessado diretamente pelo índice. Na lista, é necessário percorrer os nós anteriores até chegar ao i-ésimo nó.

6. Qual a diferença entre o operador . e o operador -> em C?

. acessa um campo diretamente de uma estrutura, enquanto -> acessa um campo através de um ponteiro para uma estrutura.

7. Por que malloc() é necessário para criar novos nós dinamicamente?

Porque malloc() reserva memória durante a execução do programa, permitindo criar novos nós conforme necessário.

8. Por que buscar um elemento pode ser $O(n)$, enquanto inserir no início é $O(1)$?

Porque para buscar pode ser necessário percorrer todos os nós da lista. Para inserir no início, basta criar o novo nó e fazer o ponteiro dele apontar para o antigo primeiro nó.

